



Sreyas Institute of Engineering and Technology

An Autonomous Institution

Approved by AICTE, Affiliated to JNTUH

Accredited by NAAC-A Grade, NBA (CSE, ECE & ME) & ISO 9001:2015 Certified

A
PROJECT
ON
DATA ANALYSIS BY WEB SCRAPPING USING PYTHON

[REVIEW-1]

BACHELOR OF TECHNOLOGY
BY
COMPUTER SCIENCE AND ENGINEERING
BY

BATCH-16

GODAVARTHY PRADYUMNA
KANKATI SAI ROHITH.
THAMMALA CHANDANA
RADHARAPU ROHIT

23VE1A05E2
23VE1A05F5
23VE1A05J6
24VE5A0519

ACADEMIC BATCH:(2023-2027)

Under the guidance of
MR.M.V.NAGESH SIR-Assistant Professor

Content:

- Abstract
- Existing system
- Drawbacks
- Proposed system
- Advantages
- Requirements
- Architecture diagram
- Queries

ABSTRACT:

- In today's technologically advanced world, large enterprises across various industries rely heavily on data for everything, from business growth to expansion. As a result, it would not be too far off to say that data is the electricity that powers all of today's industries. Large enterprises use data to improve performance, generate revenue, and provide better customer service. In fact, the automobile industry is also harnessing the power of data to improve vehicle safety. Their goal is to build powerful machines that think in data form. With that in mind, web scraping refers to the use of bots in extracting crucial data and content from websites. Web scraping is classified into three essential fragments, i.e., the web scraper drawing the desired links from the internet, using the source links to extract information, and storing the gathered information into a well-structured CSV file. Python programming language can be used to aid the process. Through linking various libraries, Python can be a valuable resource in gathering or scraping desired data from multiple targeted websites online. Consequently, this paper focuses on the efficacy of using version 3.6. /11 Python programming language in web scraping for data-centric applications.

Existing system:

The existing system “Data analysis by web scraping using python” presents a study on web scraping as a method for data extraction and Python as an effective tool for data analysis. Web scraping helps gather information from various websites for business intelligence, research, and decision-making. The study uses Scrapy (a Python web crawling framework) to automate data collection.

The existing system <http://ijream.org/papers/IJREAMV07I02SJ003.pdf> data is displayed in images (tables, graphs, charts), structured reports, or data driven insights for business.

While the display of data includes several drawbacks that need to be addressed which includes data overloading, loss of context, inconsistent formatting many more.....

Drawbacks:

1. Data Overload – Too much information can make visualization overwhelming and difficult to interpret
2. Misinterpretation – Poorly designed graphs or charts can lead to incorrect conclusions.
3. Limited Customization – Some visualization tools may not provide enough flexibility for complex datasets.
4. Performance Issues – Large datasets can slow down dashboards or reports, making real-time updates difficult.
5. Inconsistent Formatting – Data may be displayed in different formats (e.g., CSV, JSON, tables, charts), causing confusion.
6. Loss of Context – Summarized data may omit important details, leading to misleading insights.
7. Visualization Bias – The way data is presented can influence perception, leading to biased decision-making

Proposed system:

This system enhances the existing Tkinter to display the scraped and analyzed data in a Graphical User Interface (GUI). Tkinter allows you to create a simple desktop application to show the extracted data in a table or chart.

1. **Displaying Scraped Data in a GUI Table:** Instead of viewing extracted data in a CSV file, you can display it in a structured table (Treeview Widget).

✓ Example: Show product names and prices scraped from an e-commerce website.

✓ Useful for: Quick visualization of extracted data without opening a CSV file.

2. **Graphical Data Analysis (Charts & Graphs):** Scraped data can be visualized inside Tkinter using Matplotlib.

✓ Example: A bar chart showing price variations for different products.

✓ Useful for: Identifying trends in data (e.g., stock prices, weather changes). Comparing product prices from multiple websites.

3. **Real-Time Data Updates & Live Scraping:** Tkinter can be used to display real-time scraped data that updates automatically.

✓ Example: A stock market tracker that scrapes and updates stock prices every few seconds.

✓ Useful for: Monitoring live data (e.g., currency exchange rates, live sports scores). Scraping news headlines and updating the interface dynamically.

Advantages:

1. Easy to Use & Beginner-Friendly:

- Built into Python – No need to install additional dependencies.
- Simple syntax – Easy to learn and implement.
- Fast development – Quickly create interactive GUI applications.

.2. Interactive Display of Scraped Data:

- View extracted data in real-time – No need to open CSV files manually.
- Use Treeview widget – Displays structured data like a spreadsheet.
- Add search, filter, and sorting options for better analysis.

3. Real-Time Data Updates & Live Scraping:

- Tkinter can automatically refresh scraped data every few seconds.
- Use after() method to update stock prices, weather, or sports scores dynamically.

4. Data Visualization (Charts & Graphs):

- Tkinter supports embedding Matplotlib charts to visualize trends.
- Display bar charts, line graphs, pie charts, and histograms.

Software and Hardware requirements:

Hardware Requirements:

- Minimum System Requirements:
- Processor: Intel i3 (or equivalent)
- RAM: 4GB (for small-scale scraping)
- Storage: 20GB free space
- Internet Connection: Required for scraping

Software Requirements:

- Programming Languages:
- Python (preferred)
- Libraries & Frameworks:
- Web Scraping: BeautifulSoup, Scrapy, Selenium, Requests
- Data Handling: Pandas, NumPy
- Data Storage: SQLite, MySQL, MongoDB
- Data Visualization: Matplotlib, Seaborn
- Operating System: Windows / macOS / Linux

Functional and Non-Functional requirements:

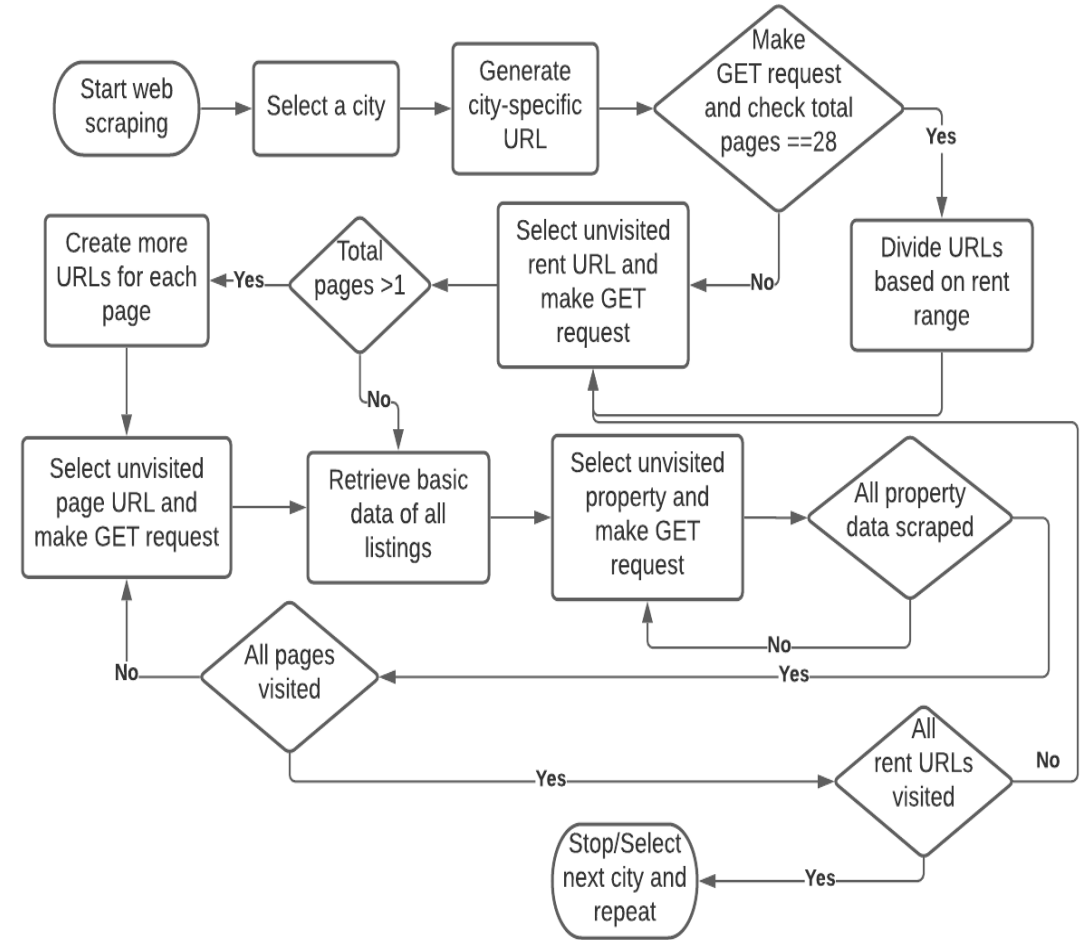
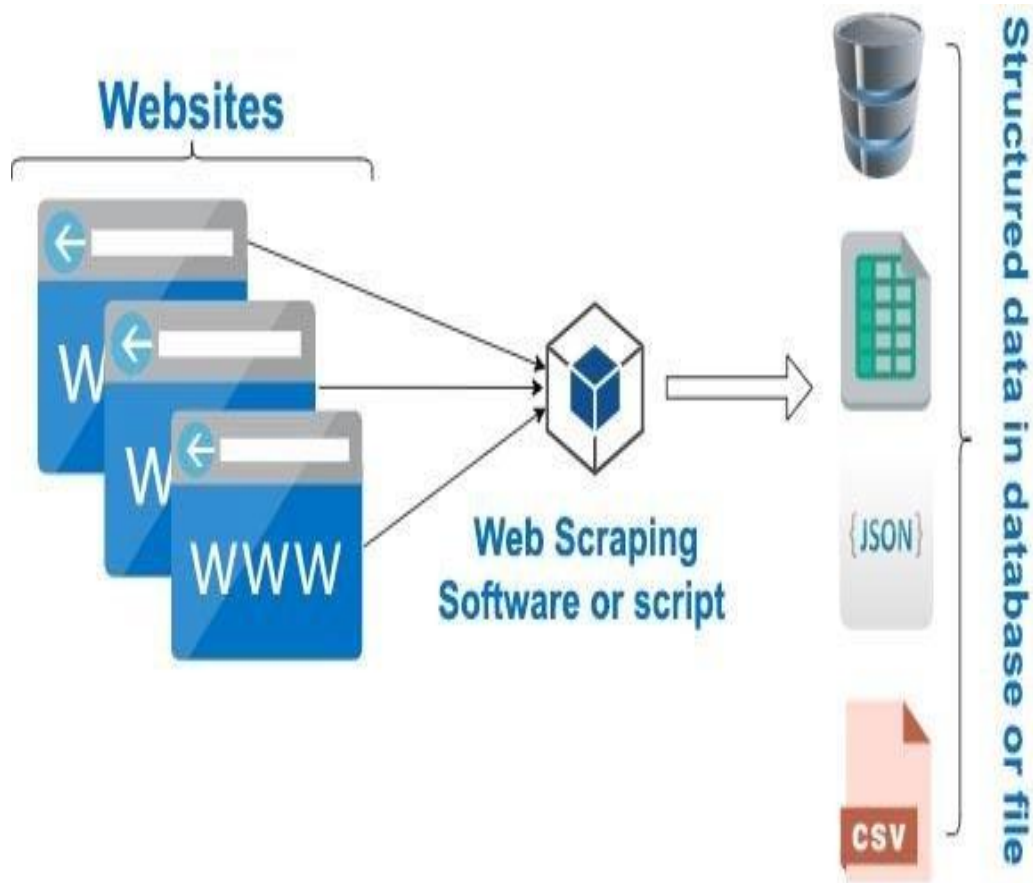
Functional requirements:-

- These define what the system should do.
- Extract data from websites using web scraping techniques.
- Parse and clean extracted data.
- Store data in structured formats (CSV, JSON, Database).
- Process and analyze data using Pandas/NumPy.
- Visualize data using charts and graphs.
- Handle dynamic websites with JavaScript-rendered content (using Selenium).
- Save scraped data in a local file or database.

Non-Functional requirements:

- These define the system's quality attributes.
- Performance: Should scrape and process data efficiently without delays.
- Scalability: Should be able to handle large datasets if needed.
- Security: Must respect robots.txt and avoid scraping sensitive data.
- Usability: Should be easy to run, with clear documentation.
- Maintainability: Code should be modular and reusable.
- Error Handling: Should handle failures like connection issues, captchas, or missing data

Architecture diagram:



QUERIES???